

Best Practices for Prompting Claude 4.5

Executive Summary

- **Role-Prompting via System Messages:** Set a clear **persona or role** for Claude in the system prompt to instantly tune its expertise and tone. This **focuses the model's responses** and improves accuracy in complex domains ¹ ². *Example Snippet – System:* "You are a meticulous financial analyst." *When to use:* Any time you need Claude to adopt specialized knowledge or style (legal, medical, etc.).
- **Explicit Instructions & Context: Be specific and provide rationale.** Claude 4.5 follows precise directions closely, but is less likely than earlier versions to "go above and beyond" unless asked ³ ⁴. State exactly what you need (format, depth, constraints) and *why*, to guide its output ⁵. *Example Snippet – User:* "Summarize the report in 3 bullet points for a busy CEO. Emphasize financial risks, as clarity on risk is crucial for decision-making."
- **Chain-of-Thought ("Think") Triggers: Encourage step-by-step reasoning** by including prompts like "think step by step" or the keyword "ultrathink", which engages Claude's extended reasoning mode ⁶. This yields more **deliberate, thorough answers**, especially for complex problems. *Example Snippet – User:* "Think step-by-step to solve this puzzle before giving the final answer."
- **Iterative Planning and Execution:** For complex tasks, have Claude **plan first, then act**. Instruct it to draft an outline or action plan and confirm it before proceeding ⁷. This two-phase approach (plan → execute) increases **reliability and coherence**, preventing hasty or disorganized outputs. *Example – User:* "Outline the steps to implement this feature before writing any code."
- **Self-Critique and Verification Loops:** Prompt Claude to **review and refine its own outputs**. After an answer or intermediate result, ask it to double-check consistency, find errors, or verify with sources ⁸ ⁹. This catches mistakes and **improves factual accuracy** in high-stakes tasks. *Example – User:* "Now critique the above analysis for any logical errors or missing information and correct them."
- **Structured Multi-Pass Workflows:** Tackle creative or lengthy projects in **deliberate passes** (outline → draft → revise). Claude 4.5 shines in long-horizon tasks if you break them into stages ¹⁰ ¹¹. Each pass can target a specific aspect (structure, style, consistency), yielding higher-quality results with continuity. *Example – User (Pass 1):* "Draft an outline for the story based on these notes." *(Later pass)* "Improve the draft by adding sensory details and checking for plot consistency."
- **Context Management: Leverage Claude's 200K token context** without overloading it at once. Feed only relevant information per turn (e.g. attach just the needed sections of a document) ¹². Claude 4.5's new *context awareness* will even try to manage token budget ¹³, but you should still **curate context** to avoid diffusion. *Tip:* Maintain a brief "**Decisions Log**" of key facts or rules and include it each time to keep Claude on track ¹⁴.
- **Formatting and Style Control:** Steer Claude's output format by **instructing what to do, not just what not to do** ¹⁵. Use structured prompts and even **XML-style tags** to enforce format (e.g. `<bullet_points>` ... `</bullet_points>`) ¹⁶ ¹⁷. **Match your prompt style** to the desired answer style (omit Markdown in your prompt if you don't want it in the answer) ¹⁸. *Example – User:* "<summary_report>Write a one-paragraph executive summary with no bullet points or markdown.</summary_report>"
- **Controlling Verbosity:** By default Claude 4.5 is **more concise** and may skip extraneous elaboration ¹⁹ ²⁰. If you need more detail or transparency, **ask for it explicitly**. Conversely, to tighten an overly verbose draft, give clear limits (e.g. "limit answer to 2 sentences" or "compress

paragraphs >120 words" ²¹). Example – User: "After completing each step, provide a brief summary of what you did and why." (to increase verbosity)

- **Agentic Tool Use & Action Bias: Direct Claude's level of proactiveness.** If you want it to take actions (in Claude Code or other agent contexts) rather than just suggest, **phrase commands imperatively** ("Change X to Y" vs "Could you maybe fix X?") ²² ²³ . You can also include a system directive like `<default_to_action>...</default_to_action>` to make it *default to taking actions* on ambiguous user intent ²⁴ ²⁵ . For a **cautious agent** that only acts when told, do the opposite with a `<do_not_act_before_instructions>` block ²⁶ ²⁷ .
- **Parallel and Sequential Task Steering:** Claude 4.5 can execute **multiple tool calls in parallel** for speed ²⁸ ²⁹ . If your use-case benefits from maximum concurrency (e.g. searching multiple files), encourage it with a prompt (see "use_parallel_tool_calls" sample) ³⁰ . If you need a more careful, step-by-step approach, instruct sequential execution with brief pauses ³¹ .
- **Minimizing Hallucinations:** For analytical or coding tasks, explicitly tell Claude **not to guess or invent facts/code without checking**. For example, instruct it to read files or documentation *before* answering questions about them ³² . This leverages Claude's willingness to use tools/operations and yields **grounded answers** ³² . Example – System:
"*<investigate_before_answering>Never answer about code you haven't seen. Always open and read the file first, or say you need it.</investigate_before_answering>*"
- **Safety-Conscious Prompting:** Claude 4.5 has stronger safety filters (AI Safety Level 3), which occasionally flag benign requests ³³ . To avoid unwarranted refusals, **frame sensitive queries with context and intent**. Emphasize a responsible approach to the topic and set boundaries in your prompt ³⁴ . If it still refuses on allowable content, try breaking the task into a **planning step and a generation step** (first ask for an outline or approach, then proceed to the detailed request) ³⁴ . Example – User: "We're writing a historical scene with battle violence. We will keep it PG-13 and focus on the characters' emotions and consequences. Outline how the scene could unfold."

(Each technique above is expanded in detail in the Best-Practices Library below, with usage examples and pitfalls.)

Release Context: What's New in Claude 4.5 (Sep 2025)

Claude 4.5 (code-named "**Sonnet 4.5**") represents a significant upgrade over its Claude 2 and Claude 4 predecessors, especially for **long-form and agentic tasks**. Announced in late September 2025 ³⁵ ³⁶ , Claude 4.5 introduced major enhancements:

- **Extended Task Endurance:** The model can maintain focus autonomously for vastly longer sessions. Internal tests showed Claude 4.5 coding continuously for **30+ hours** on a complex project (up from ~7 hours on Claude 4) ³⁷ ³⁸ . This endurance enables **multi-step problem solving** and lengthy tool-assisted workflows without losing context or requiring restarts.
- **Agentic & Tool Use Improvements:** Claude 4.5 is **better at using computers and tools** like a human assistant. It scored **61.4%** on the OSWorld benchmark for operating system tasks (versus ~42% by the previous model) ³⁹ ⁴⁰ . It now more reliably uses APIs, executes code, and navigates apps when instructed. Anthropic even built in features like a native VS Code extension and an **Agent SDK** to facilitate complex tool orchestration ⁴¹ ⁴² .
- **Longer Context and Memory:** The default context window is **200k tokens**, allowing huge documents or conversations. Anthropic also tested up to **1 million tokens** in a preview setting ⁴³ ⁴⁴ . New **context editing** and **memory tools** let Claude manage and prune its context during extended sessions ⁴⁵ ⁴⁶ . This helps it handle continuous work on extensive projects (writing a book, analyzing large datasets, etc.) with less drift.
- **Conciseness and Alignment Changes:** Claude 4.5's **communication style is more concise and grounded** than prior Claude models ⁴⁷ . It provides factual progress updates instead of overly

verbose or “self-congratulatory” commentary ⁴⁸. This was a deliberate alignment choice to focus on efficiency and reduce irrelevant verbosity. Users coming from Claude 2 may notice Claude 4.5 will not “go the extra mile” with creative flourishes unless prompted – you now ask explicitly for additional detail or creativity ³. At the same time, Anthropic touts Claude 4.5 as **their “most aligned” model** to date ⁴⁹, with AI Safety Level 3 safeguards. It is *less likely to produce disallowed content*, though the stricter filters mean it occasionally refuses borderline requests that earlier models might have answered ³³.

- **Domain Skills & Knowledge:** Beyond coding, 4.5 showed **strong gains in reasoning for finance, law, medicine, and STEM** topics ⁵⁰. Experts found it more knowledgeable and accurate in domain-specific tasks than Claude 2 or 4.1 ⁵⁰. Hard evaluation benchmarks (e.g. complex legal analysis tasks) also improved, indicating the model can follow intricate instructions and maintain context in specialized areas ⁵¹ ⁵².
- **Use Cases – Focus on Enterprise:** The launch signaled a shift toward business and enterprise use. Claude 4.5 is being integrated into products like **Microsoft 365 Copilot’s “Agent Mode”** for Office apps ⁵³, highlighting its capacity to **function as a background assistant** handling delegated tasks. Anthropic has emphasized reliability over flashy demos – aiming for AI that can **consistently carry out lengthy, multi-step workflows** for professionals ⁵⁴. For developers, the Claude API uses the model ID `"claude-sonnet-4-5"` and pricing remained the same as Claude 4 ⁵⁵ ⁵⁶, making upgrading straightforward.

Implications: Many best practices from earlier Claude versions still apply, but some need tweaking. Because Claude 4.5 is **more concise and tool-oriented by default**, prompt engineers now explicitly request any extra verbosity or creative flair that previous models might volunteer. The greatly expanded context window and improved planning mean users can attempt much larger or longer tasks – but it’s still crucial to structure those tasks (the model will try to manage context, but giving it too much at once can reduce quality). The next sections detail prompting techniques that **leverage Claude 4.5’s new strengths** (like long-horizon thinking and agent use) while mitigating its quirks (like cautiousness and brevity).

Best-Practices Library

Below is a library of prompting techniques for Claude 4.5, organized by theme. Each technique includes a description of its value, how it works, when to use it, pitfalls to watch out for, and example prompt snippets (for system and user messages).

1. Role Prompting (System Personas)

Value: Sets a **domain-expert persona or role** for Claude, dramatically boosting relevant accuracy and tailoring the tone to your needs ¹ ². Turning Claude into a *virtual specialist* (e.g. lawyer, software engineer, therapist) guides it to generate more in-depth and context-appropriate responses.

Mechanism: Provide a concise role description in the **system message** (Anthropic’s API `system` parameter). This primes the model with background knowledge and perspective. Anthropic calls this *“role prompting”* and notes it’s the most powerful use of the system prompt ⁵⁷. For example: *“You are a veteran trial attorney known for detailed contract analysis.”* All further outputs stay in character unless overridden.

When to Use: Whenever you need **expert-level output or a specific voice**. In complex tasks (legal, medical, financial analysis), role prompts yield **enhanced accuracy and focus** ¹. Also use to match a desired style or tone (e.g. friendly tutor vs. terse analyst) ⁵⁸. It’s helpful at conversation start, but you can also adjust the role mid-way if the task changes.

Pitfalls: A role prompt that's too narrow or unrealistic can misfire – e.g. claiming the AI is a famous person or an unethical role may hit policy limits or cause refusals. Keep roles **professional and task-focused** (avoid “You are Elon Musk...” or anything violating persona guidelines). Also, don't rely on role alone for instructions: provide task details in the user prompt. Finally, if the user's queries conflict with the role, Claude might get confused – you may need to remind or reassert the role if the conversation goes off-track.

Snippet (System): *“You are a senior data scientist at a Fortune 500 company, expert in statistical analysis and business KPIs. You speak in a concise, insights-driven manner.”*

Snippet (User): *“Analyze the attached sales dataset for anomalies and trends. What would a data scientist notice first?”*

2. Explicit Instruction with Context and Constraints

Value: Maximizes Claude's compliance by **clearly specifying the task, desired output format, and any constraints** (length, style, scope) *and providing context or rationale*. Claude 4.5 excels at following detailed instructions ³ ⁵, so being explicit leads to more on-point results. Including the **“why”** behind instructions can further focus the model ⁵.

Mechanism: Spell out the request with concrete details. Rather than a vague “Please summarize this,” say **“Summarize the following report in 3 bullet points, each ≤20 words, focusing on revenue changes.”** If certain content should be excluded or emphasized, mention it. *Contextualize* the request by explaining its purpose: e.g. “Summarize for a CEO who needs the key takeaways quickly.” This motivates Claude to prioritize correctly ⁵⁹. Claude 4.5 was trained to infer goals from explanations, so it will generalize instructions if you give a reason ⁵⁹.

When to Use: Always, by default. This is a general best practice heightened in Claude 4.5 because the model no longer “over-delivers” by itself as much. If you noticed earlier Claude sometimes gave exhaustive answers unprompted, 4.5 will wait for you to request that extra detail ³. Use explicit instructions especially when you need a specific format (JSON, a table, bullet list), a certain tone (formal vs casual), or strict limits (like word count or avoiding certain phrases).

Pitfalls: Over-constraining can make the response feel robotic or cause it to miss the bigger picture. Provide enough flexibility if creativity is needed. Also, long lists of instructions can sometimes lead to *conflicting* guidance – prioritize and keep them logically ordered (e.g. don't bury an important requirement at the end of a long prompt). If Claude seems to ignore an instruction, check if it conflicts with higher-level role or safety rules (some style choices might be overridden by the model's alignment/safety preferences).

Snippet (User): *“Draft an email responding to this customer complaint (below). Use a polite, apologetic tone and provide a clear resolution. The email should start with a brief apology, then explain what went wrong in one sentence, and offer a refund or alternative. Do not use any technical jargon or blame the customer. Keep it under 150 words.”*

(Include the customer complaint text as context if available.)

Snippet (Rationale Addition – User): *“The goal is to retain the customer's trust, so the tone must be reassuring and solution-focused.”*

3. Chain-of-Thought Prompting (“Let Claude Think”)

Value: Improves reasoning on complex problems by prompting Claude to break down its reasoning process into steps or reflect silently before answering. Claude 4.5 can perform extended “thinking” when asked, leading to more logical and correct solutions for math, puzzles, code, or multi-part questions ⁶. It reduces errors by making the model **justify or plan** its answer internally (or even explicitly in the output, if desired).

Mechanism: Use trigger phrases and instructions that encourage step-by-step reasoning. Phrases like “Think step by step”, “Let’s think this through:”, or “**Consider this carefully before answering**” cue the model to engage its chain-of-thought mode. Notably, Anthropic built an “**extended thinking**” feature into Claude 4 models – certain words will invoke deeper computation time ⁶. For example, saying “*ultrathink about the problem and list your steps*” signals Claude to apply maximum deliberation ⁶⁰. You can also explicitly request the reasoning either *before* the final answer (e.g. “Show your reasoning then conclude”) or *hidden* (e.g. “Think through silently and just give the answer”).

When to Use: For **complex analytical tasks** (multi-step math, programming logic, troubleshooting, logical puzzles) and whenever you suspect a direct answer might be flawed. It’s also useful when you want transparency in the model’s reasoning (you can ask it to output its thought process). If an answer seems too pat or incorrect, follow up with “*Please explain how you got that*” – often Claude will reveal a mistake in its reasoning that you both can then address.

Pitfalls: If you ask for chain-of-thought **in the output**, remember the model will make its reasoning readable, which might slow it down or introduce unnecessary detail for simple tasks. For very straightforward queries, this can be overkill. There’s also a slight risk of *exposing the model’s internal logic that might contain errors*: if it reasons incorrectly, it could still lead itself to a wrong answer. Always verify the final answer. Additionally, avoid instructions that the model “think *silently* and not show the steps” *and* also expecting verification – if you want to inspect reasoning, better to have it printed. Lastly, keep chain-of-thought requests aligned with the task (don’t force a needless step-by-step on creative writing, for example, as it might stifle the creativity).

Snippet (User): “Solve this problem step by step. First, enumerate the steps you will take, then give the final answer:\nQ: A train travels 100 km at 50 km/h and then 100 km at 100 km/h. What’s the average speed?”

Snippet (System or User): “Before answering, think carefully about each stage of the analysis. You can do calculations or reasoning in detail. Only then provide your conclusion.”

Snippet (Using “ultrathink”): “This is a tough question – ultrathink and consider all possibilities before you respond.”

4. Iterative Planning then Action (Two-Stage Approach)

Value: Improves task organization and outcome quality by first having Claude produce a plan, outline, or strategy, and only then execute or answer. This mimics a human expert’s approach to complex tasks: plan first, then implement. It prevents haphazard answers and allows you to **verify or tweak the approach** before final output.

Mechanism: Prompt Claude to **create a structured plan** or list of steps *without yet solving*. For instance: “Outline your approach to diagnosing this issue before proceeding” or “List the steps you will take to write the code, then await confirmation.” Once the plan is presented, you (or your system

program) can approve it or modify it, then instruct Claude to carry out the plan step by step. Claude 4.5 is capable of holding the plan in memory and following it, especially if the steps are enumerated clearly. Some users explicitly ask Claude to output a plan and **stop** – using commands like “Do not start writing the final answer yet.” When ready, a simple “Looks good, proceed with step 1” can trigger execution.

When to Use: **Coding tasks** (generate an algorithm design before coding), **long-form writing** (outline chapters before writing full text), **multi-step data analysis or troubleshooting** (plan an investigation procedure), and any scenario where the solution space is large or ill-defined. It’s particularly powerful in agentic coding with Claude: asking for a plan ensures the AI doesn’t immediately begin editing files or calling tools blindly.

Pitfalls: It requires an extra step, so it can slow things down slightly. In some cases, Claude’s plan might be obvious or overly general, adding little value – use this technique when the task truly merits forethought. Also be aware that if the conversation is not strictly managed, Claude might start executing the plan immediately unless you *clearly* instruct it to wait (Claude usually follows the “don’t act before confirmation” if told, but if you omit that, it might jump in). When implementing the plan, if something changes, be prepared to update the plan or have Claude revise it. Finally, planning doesn’t guarantee correctness – a flawed plan will yield a flawed result, so review those steps.

Snippet (User): *“Our goal is to refactor this legacy function. First, provide a step-by-step plan for the refactor (don’t make changes yet). After we discuss the plan, we’ll proceed.”*

Claude (Assistant) might reply with a plan, e.g.: 1. Understand current function behavior from `analytics.js`.
2. Write unit tests covering expected outputs.
3. Modularize large function into smaller ones... (etc.)

Snippet (User follow-up): *“Great, proceed with step 1 and 2 and show me the results, then we’ll continue.”*

(Here the user is approving and guiding Claude through the plan steps sequentially.)

5. Self-Critique and Correction

Value: Harnesses Claude’s ability to **analyze and improve its own answers**, leading to more accurate and refined results. By asking Claude to review its output, you can catch errors, inconsistencies, or missing pieces that the initial pass might have overlooked ⁸ ⁹. This effectively gives you a **built-in editor or double-checker**.

Mechanism: After Claude produces an answer or solution, prompt it with something like: “Do you see any mistakes or areas to improve in your above answer? If so, correct them.” or “Now critique your solution step by step.” Claude will then re-read (it has the conversation context) and often spot logical mistakes, calculation errors, or simply polish wording. Another strategy is to explicitly request checks: e.g. “Cross-verify the answer using a different method” or “List assumptions in your solution and verify if they hold.” In coding, you might ask Claude to run tests or static analysis on the code it just wrote (if using Claude Code with tools). This turns the session into a feedback loop where Claude iteratively improves its output.

When to Use: In **important or sensitive tasks** where correctness matters (e.g. medical or legal summaries, mathematical derivations, code for production). It’s also useful if the first answer seems off – rather than throwing it away, get Claude to diagnose the issue. Claude 4.5’s longer context means it

can reconsider its answer in light of the entire conversation or additional info. Use it for any answer that would benefit from a second pass: complex essays (ask for improvements in clarity or style), long calculations (check each step), etc.

Pitfalls: Claude can sometimes be *over-critical* or make unnecessary changes if the original answer was actually fine. It might also incorrectly label something as an “error” due to overanalysis. So, treat its self-critique as suggestions – verify the changes. There’s also a chance that if Claude truly doesn’t know the subject well, it may not catch its mistakes. In those cases, external validation (by a person or another tool) is still needed. Also, avoid infinite loops of self-correction: one or two iterations are usually enough. If Claude keeps toggling back and forth on an answer, that’s a sign to intervene or break the problem down further.

Snippet (User): *“Review the above answer. Identify any factual inaccuracies or reasoning errors and then provide a corrected answer.”*

Snippet (Assistant response): *(Claude might say something like:)*

“Upon review, I noticed a mistake: I assumed X, but in the text it was actually Y. Also, point 3 could be clearer. Here is the revised answer:” ... (new corrected answer) ...

Snippet (User, alternative approach): *“Double-check your calculations in the solution. Are the numbers and logic consistent? If not, fix them.”*

6. Multi-Pass Drafting & Refinement

Value: Produces **higher-quality creative or long-form outputs** by splitting work into multiple focused passes instead of one giant prompt. This plays to Claude 4.5’s strength in maintaining long-term consistency and allows targeted improvements on each round ¹⁰ ⁶¹. The end result is often more coherent and polished than a single-pass generation.

Mechanism: Draft in stages. For example, in a writing task: first ask for an outline, then flesh it into a rough draft, then do a series of refinement prompts (fix continuity issues, enhance style, tighten length, etc.). Each pass has a specific goal. You effectively create a **feedback loop** where Claude uses the previous output as input for the next refinement. Claude 4.5 can handle this especially well thanks to its long context (you can keep the whole draft and outline in context for revision). The key is to guide each pass with clear instructions: e.g. *“Now, improve the character dialogue to match the tone of the earlier chapters.”* or *“Next pass: check for any contradictions with the summary in Chapter 1.”* This method is heavily used in the creative writing workflow for Claude 4.5 ¹¹ ⁶², but equally applicable to plans, reports, etc.

When to Use: Long documents (stories, articles, reports), **complex coding tasks** (iteratively improving code or adding features), or any output that benefits from revision (marketing copy, design documents). Also use it when you have a decent draft but want to explore different versions – you can branch into multiple refinement paths (e.g. *“In one version, make it formal. In another, make it humorous.”*). Multi-pass prompting is great when you need to satisfy multiple constraints (you can address them one at a time in successive passes).

Pitfalls: It takes more interaction and careful tracking of changes. You should maintain a *“decision log”* or remember what changes were made in each pass to avoid undoing them later ¹⁴. Claude can sometimes introduce new errors while fixing others, so keep an eye on unintended changes. There’s also the risk of **model drift** over many passes – e.g. wording might slowly change style. Mitigate this by

re-providing the original style examples or key constraints occasionally ("here again is the style guide – ensure the revision adheres to it"). Also, ensure you don't exceed the context limit with all these iterations; prune older parts if needed or use the new context editing features to maintain focus.

Snippet (User – Pass 1): *"Outline the key points for an essay about AI in education."*

(Claude outputs an outline.)

Snippet (User – Pass 2): *"Great. Now write a first draft based on this outline (aim ~500 words). Don't worry about perfect wording yet – just cover all points."*

(Claude outputs a draft.)

Snippet (User – Pass 3): *"Next, refine this draft: ensure the tone is formal and academic, and add transitions between paragraphs. Also, highlight any assumptions that need citations."*

(Claude revises accordingly.)

Snippet (User – Pass 4): *"Final pass: shorten the introduction by removing fluff, and correct any grammar issues. Then provide the final polished essay."*

7. Few-Shot Examples for Guidance

Value: Demonstrates the desired output or approach by example, which Claude can then mimic. By providing one or more examples (prompts and ideal responses), you leverage Claude 4.5's powerful pattern-matching on those examples to produce answers in the same style or format ⁶³. This technique is especially useful for complex formats or specialized style that are hard to describe in words alone.

Mechanism: In your prompt, include a **sample Q&A pair or formatted text** as a guide. This can be done in the system prompt or user prompt. For instance, if you want Claude to output in a specific JSON schema, show a short example of the schema with dummy data. If you need a particular tone, you can give an example user request and an ideal assistant answer with that tone (few-shot dialog). Claude will infer the rules from the examples. With Claude 4.5's large context, you can supply fairly extensive examples if needed. Anthropic's guidance suggests ensuring examples **closely align with the task** you want ⁶³ – the model will pick up on patterns, so make sure they reflect what you want (and not what you *don't* want).

When to Use: When formatting is critical (e.g. output must be valid code, or a certain markup), when style is critical (e.g. mimic a certain writing voice or jargon level), or when instructive attempts have failed. Also use it to clarify an ambiguous task: sometimes showing is easier than telling. For classification or extraction tasks, providing a couple of manually worked examples in the prompt can significantly boost accuracy.

Pitfalls: Example messages count toward context tokens, so don't overdo it if not necessary. The model might also become too **anchored** to the examples, sometimes copying irrelevant specifics (like similar content or wording from the illustration). Prevent this by using placeholders or generic content in examples. Also be wary of inadvertently biased examples – the model may carry those biases or errors forward. Finally, if the examples are too complex or too many, it could confuse Claude about

what you want (especially if they cover diverse styles). Generally keep few-shot examples to 1–3 well-chosen instances.

Snippet (System or User with example): *“Example – Q: Describe the impact of AI on healthcare in one sentence.\nA: AI is revolutionizing healthcare by enabling early disease detection and personalized treatment plans.\nNow your turn: Q: Describe the impact of AI on education in one sentence.”*

(Claude will infer it should respond with a similarly structured one-sentence answer.)

Snippet (format example): *“Format the output as CSV. Example: ‘Name,Age\nAlice,30\nBob,25’. Now provide the data in CSV for the following entries:...”*

8. Control Output Verbosity and Detail

Value: Lets you **tailor the length and depth** of Claude’s answer, from terse summaries to elaborate analyses. Claude 4.5 defaults to an efficient brevity ¹⁹, but you can override this to get more explanation or, conversely, enforce conciseness. This ensures the response is as detailed (or succinct) as you need.

Mechanism: **Instruct desired length or level of detail** explicitly. For more verbosity: ask for a detailed answer, or specify sections (e.g. “Provide a detailed explanation after the solution”). Phrases like “*explain like I’m 5*” or “*walk me through it step by step*” prompt longer answers. You can also set a minimum length: e.g. “*Write at least 4 paragraphs on...*”. For brevity: request a summary or give a constraint: “*in 50 words or fewer*”, or “*just a yes or no with one sentence justification.*” Claude will usually respect explicit length limits or at least come close. Another method is using the `max_tokens_to_sample` parameter via API to hard-limit length (but that’s outside prompt content control). Additionally, Claude 4.5 tends not to repeat itself or add fluff, but if you find it verbose, you can reinforce: “*Be concise.*” or use an imperative like “*Answer in a single sentence.*”

When to Use: **Steer verbosity** whenever the default might not match your needs. If you notice Claude giving too high-level an answer, ask for more depth. For instance, in brainstorming, you might say “*Give me 10 ideas with a short explanation for each.*” In contrast, in a quick Q&A or when integrating into a UI with space constraints, you might always include “*Keep your answer under 3 sentences.*” Also use it in multi-turn interactions: if Claude’s first answer is too brief, you can follow up: “*Could you expand on that, especially the second point?*” – it will elaborate.

Pitfalls: Setting extreme brevity can omit important details – if you say “one sentence” for a complex question, expect simplification. On the flip side, asking for extreme verbosity can lead to rambling or redundancy. Also note: Claude will try to obey length guidelines but not at the cost of completely malformed answers; sometimes it might use two sentences when told one, to be grammatically correct or cover a crucial point. Treat length instructions as guidelines, not absolute in all cases. If it’s critical (e.g. fitting a social media character limit), you may need to double-check and ask for revision if it’s too long.

Snippet (User – verbose): *“Explain the theory in detail. Provide at least three paragraphs with examples for clarity.”*

Snippet (User – concise): *“In one sentence, what does this data imply about our sales trend?”*

Snippet (User – list with detail): *“List 5 key findings from the report, each in 1-2 sentences.”*

9. Format and Style Steering with Tags and Scaffolds

Value: Achieves **precise control over output format and style**, which is crucial for structured data outputs (JSON, XML), markdown generation, or stylistic consistency. By using special **XML-like tags or section headers** in your prompt, you clearly delineate instructions and desired format, making it easier for Claude to comply ⁶⁴ ¹⁷. Claude 4.5 has been trained to recognize and follow such patterns.

Mechanism: Wrap specific format instructions in a unique tag or use headings. For example, Anthropic suggests tags like `<code_output>...</code_output>` around instructions to produce code without explanation, or as given in docs: `<avoid_excessive_markdown_and_bullet_points>...</avoid_excessive_markdown_and_bullet_points>` with detailed formatting rules inside ¹⁷ ⁶⁵. The model will interpret these as directives and shape its output accordingly, often without literally including the tags in the final answer (it knows those were meta-instructions). You can also section your prompt into parts like **"Context: ..."**, **"Task: ..."**, **"Constraints: ..."**, **"Output Format: ..."**. Claude responds well to **structured prompts** – it tends to mirror the structure in the answer ⁶⁶. For style, providing a snippet of desired style (see *Few-Shot Examples* above) or explicitly stating "Use a *neutral, factual tone*" or "Respond in YAML format" does the trick. If Claude's output still deviates, double-check that your prompt isn't inadvertently formatted in that style (because it sometimes mimics the prompt's formatting quirks).

When to Use: When you **must have a specific format** (e.g., an API expects JSON), or you need to minimize later editing. It's also useful for long instructions: isolating them in tags or a clearly separated block can help Claude parse your prompt without confusion. Use it for complex outputs like tables, code, or whenever you noticed Claude giving undesired formatting – you can correct by explicitly instructing the format next time.

Pitfalls: Over-using XML tags for everything might make prompts cumbersome. Also, tags are not a standard language feature – they rely on the model's training signals. Claude generally respects them, but there's no guarantee another model would. Ensure the tags you choose aren't conflicting with actual content (don't use `<summary>` if your content also includes HTML/XML, for instance). Another minor pitfall: sometimes Claude will literally output the tags if it thinks they're part of the required answer (especially if you use common words). Using a somewhat unique tag name (like `<analysis_response>` rather than `<response>`) can avoid confusion.

Snippet (User with tags): `<recipe_output>Please output the recipe as JSON with keys "ingredients", "steps", and "total_time". Use an array for ingredients and step-by-step instructions for steps.</recipe_output>`

(Claude will likely return just the JSON object, no explanatory text.)

Snippet (Structured prompt):

```
Context: The user manual text is provided.
Task: Extract all error codes and their meanings.
Output Format: Markdown table with two columns: "Error Code" and
"Description".
```

"Now, given the above, produce the output."

(Claude would then produce a Markdown table if the data was in context.)

10. Default Action vs. Suggestion (Agentic Bias Control)

Value: Controls whether Claude acts autonomously or waits for explicit confirmation when operating as an agent. In coding or tool-using modes, sometimes you want Claude to **take initiative** (fix code, execute changes) rather than just suggesting – or vice versa, to be cautious. This technique helps align Claude’s behavior with user expectations, **making it either more proactive or more consultative** ²²

²³ .

Mechanism: Wording of the user request is key. Imperative verbs (“do X”, “change Y”) lead Claude to take action, while questions or tentative language (“could you maybe...”) lead to advice. Anthropic documentation shows that phrasing matters: “Can you suggest improvements?” yields suggestions, whereas “Improve this code by doing X.” yields direct changes ²² ²³ . To systematically enforce this, you can include a system directive block as mentioned earlier: `<default_to_action> ... </default_to_action>` to tell Claude to assume it should act by default ²⁴ . Conversely, a `<do_not_act_before_instructions>` system note will make it hold off and always explain or ask before acting ²⁶ ²⁷ . This is particularly useful in environments like Claude Code or any autonomous agent setup where you either want hands-off automation or a human-in-loop style.

When to Use: Developer and agent scenarios. If using Claude to edit files, run commands, etc., decide upfront how aggressive it should be. For a fully automated agent, lean proactive: instruct it to implement changes unless told otherwise (so you don’t have to green-light every step). For a code assistant working with you, maybe lean conservative: you might want to review suggestions before it applies them. Also use this in conversational tasks – e.g., a customer support bot might be set to either directly resolve issues (offer refunds, etc.) or just provide options and let a human decide. Outside of agent use, this principle still applies: phrasing any request as a direct command tends to yield direct answers, whereas asking for suggestions yields milder responses.

Pitfalls: If Claude is set to *always act*, it might occasionally do something you didn’t intend (because it “inferred your intent” incorrectly). Monitor initial runs carefully. On the flip side, if it’s too hesitant, it may keep asking for confirmation (“I suggest doing X, should I proceed?”), which can be inefficient. You may need to adjust the directives if you find the balance isn’t right. Also, be mindful of safety – a proactive Claude will still refuse disallowed actions, but could push boundaries. Keep its autonomy within tasks you’re comfortable automating.

Snippet (System – proactive):

```
<default_to_action>
By default, carry out the user's requests directly rather than just
suggesting. If the instruction is ambiguous, make a reasonable assumption
and proceed with an action.
</default_to_action>
```

Snippet (System – cautious):

```
<do_not_act_before_instructions>
Do not execute changes or final actions unless the user explicitly confirms.
```

```
If a request is unclear, provide options or information rather than taking
action.
</do_not_act_before_instructions>
```

Snippet (User phrasing – suggestion vs action): - *Suggestion style: “What changes would improve the clarity of this paragraph?” (Claude will likely suggest edits.)*

- *Action style: “Edit the paragraph to improve clarity.” (Claude will likely rewrite the paragraph outright.)*

11. Parallel vs. Sequential Tool Use

Value: Optimizes how Claude uses tools or performs sub-tasks by controlling **concurrency**. Claude 4.5 can execute multiple operations in parallel (e.g. read several files simultaneously, make concurrent web searches) which can greatly **speed up tasks** ²⁸ ²⁹. However, sometimes a sequential step-by-step approach is safer. This technique lets you influence that behavior to match the task's needs.

Mechanism: Use explicit instructions about performing actions in parallel or one at a time. Claude's default is to be fairly aggressive with parallelism when it sees fit (it might try to fetch multiple resources at once) ⁶⁷. To **ensure parallel execution**, you can say: “Do X, Y, and Z in parallel if possible.” Anthropic's example system tag `<use_parallel_tool_calls>` provides a template: it tells Claude to call tools simultaneously for independent tasks ³⁰. Conversely, to force sequential operation, instruct something like: “Complete each step one at a time, and only start the next after finishing the previous.” You might also mention adding a small delay or check between steps for safety (though Claude itself can't delay time, it will conceptually pause if waiting on results). In code agent use, if you notice Claude spawning too many threads or processes, rein it in with a sequential directive.

When to Use: **Research and data gathering tasks** (parallel web searches), **batch file operations** (reading many files), or anytime speed is more important than step-wise transparency. Use sequential mode for **fragile operations** that could interfere with each other or when debugging (one change at a time). Also, if the system running Claude has limited resources, you might prefer sequential to avoid overload (Claude can be *too* aggressive, e.g. launching many processes at once ⁶⁷, which could strain a system). In multi-user settings or when actions have side effects, sequential might prevent conflicts.

Pitfalls: Running everything in parallel could make it harder to pinpoint issues (if something goes wrong, several actions were in flight). Also, if tasks actually depend on each other's results and you forced parallel, Claude might end up guessing some dependencies. Conversely, sequential execution can be slower for large tasks. One subtle pitfall: Claude might sometimes decide not to fully parallelize even if told (perhaps it sees a hidden dependency), so don't assume parallel instruction is always literal – but generally it tries. And note, these instructions matter only if using Claude in a context where it *can* do parallel actions (like Claude Code with tool use). In pure QA conversations, this is less applicable except conceptually (e.g., thinking of multiple ideas in parallel, which is abstract).

Snippet (System – maximize parallel):

```
<use_parallel_tool_calls>
If multiple independent tasks can be done, do them all at once. For example,
if you need to read file1, file2, and file3, read them in parallel rather
than one by one.
</use_parallel_tool_calls>
```

Snippet (User – enforce sequential): *“Complete the following steps one at a time, in order, and confirm after each before moving on: 1. Compile the code. 2. Run the tests. 3. Generate a report. Do not overlap these steps.”*

12. Memory and Context Management Techniques

Value: Helps Claude manage large contexts over long sessions by **externalizing memory** or informing Claude about context handling. This prevents the model from losing important info in a sea of text, and avoids running into token limits unexpectedly. Essentially, it **keeps Claude’s “mind” organized** in extended interactions.

Mechanism: One approach is to use **persistent summary or memory files**. For example, in Claude’s coding environment, a special file `CLAUDE.md` can hold project knowledge (architecture, style guides, key decisions) which Claude always loads ⁶⁸ ⁶⁹. In general usage, you can simulate this by maintaining a running summary and re-providing it as needed. Another tactic: periodically ask Claude to **summarize the conversation so far** or note “what it has learned” and then supply that summary in a system message for subsequent turns (this acts like an external memory). Claude 4.5 even has a *context editing* ability where it can drop or compress its context – you can encourage this by saying: *“We’ve discussed A, B, C. You can forget the details of A now and focus on B and C going forward.”* Additionally, if using an agent loop, explicitly tell Claude about context resets: e.g. *“After each round, I’ll provide a summary of relevant info so far. Use that to continue.”* Claude is also aware of its token budget to an extent ¹³; you can reinforce by warning it if a lot of data is about to be dumped in: *“The next message contains a lengthy document. Skim it and extract only the needed facts to conserve tokens.”*

When to Use: Long chats (dozens of turns), complex projects (coding over multiple files and sessions), or any time you see Claude starting to forget earlier details or contradict them. If you plan on a conversation spanning hours or days (with the 5-hour session limit in mind ⁷⁰), memory techniques are essential. Use it in creative writing too: e.g. keep a **“Facts & Characters” list and prepend it occasionally** to avoid drift over chapters. Also if collaborating with Claude over time, the `CLAUDE.md` concept (project-wide persistent instructions) is very powerful for keeping consistency across sessions ⁷¹ ⁶⁹.

Pitfalls: Manual memory management is effort. Summaries can accidentally omit something crucial, so carefully curate them. Also, if you provide a summary that contains an error, Claude will propagate that – so occasionally verify the memory. Another caution: pushing too much into context under the assumption “Claude can handle 200k tokens” can degrade performance – the model might lose focus. It’s still wise to keep only relevant info. Finally, be mindful of privacy and safety when constructing memory prompts (don’t put anything disallowed in a `CLAUDE.md` or summary thinking it’s invisible – it’s treated as user-provided content).

Snippet (Maintaining a Decisions Log in prompt): *(At conversation start or as a system message):*
“Project Facts (to remember): - Our product’s codename is Delta, which means any mention of ‘Delta’ refers to the same project. - Client prefers formal tone. - Previous conclusion: Hypothesis A was proven false, we are pursuing Hypothesis B.”

(Claude will keep these in mind; you can update this list and resend as needed.)

Snippet (User asking for summary): *“Before we continue, summarize the key points we’ve established so far in 5 bullet points.”*

(Claude provides summary; you can then use or refine those bullet points in subsequent prompts.)

Snippet (Claude.md usage in Claude Code): Include guidelines and context in a `CLAUDE.md` file, for example:

```
# Code Style
- Follow PEP8 for Python.
- All functions must have type hints.

# Key Variables
- `customer_data`: global dict storing user profiles.
```

(Claude will automatically read this at session start and comply without needing it in each prompt.)

13. Research & Fact-Checking Prompts

Value: Instructs Claude on **how to gather and verify information**, yielding more accurate and credible answers in tasks that require external knowledge or source citation. Claude 4.5 has improved “agentic search” abilities ⁷², so guiding it to research methodically can leverage that strength and minimize hallucination.

Mechanism: If connected to tools (like a web search API or knowledge base), tell Claude to use them instead of guessing. Even without actual tools, you can simulate a research approach: e.g. *“Brainstorm possible answers, then check each against the data provided.”* A particularly powerful pattern is to have Claude generate **multiple hypotheses or answers and then compare them**. For instance: *“Come up with two possible solutions and then explain which is more likely and why.”* Another technique: explicitly request **source attributions**. Claude can provide source-like references if asked (though it doesn’t have an internet connection by default, it was trained on citation styles). For example: *“Answer with facts and cite the document name for each fact.”* This pushes the model to be mindful of origin of information. Anthropic suggests encouraging cross-verification ⁷³ – e.g. *“Consult multiple sources and ensure they agree before finalizing your answer.”* In a tool-enabled scenario, you might instruct: *“Use the web search tool and open the top 3 results on X, summarize the findings from each, then synthesize an answer.”*

When to Use: **Open-ended questions, knowledge queries, analytical report generation**, or whenever you suspect the information might be incomplete or subject to error. Also if the answer needs to be **defended or explained with evidence** (e.g. a summary for an article – you want to ensure key points are covered correctly). This is especially relevant for high-stakes factual responses (medical, legal, scientific queries) – you want Claude to double-check itself via the data provided or logical consistency.

Pitfalls: If not actually connected to a live knowledge source, Claude’s “research” is limited to what it already knows (which might be outdated or incomplete). It might fabricate sources or citations if pressed too hard (“According to [nonexistent] study...”). To mitigate, if you gave it documents, ask it to reference those by name or number. If not, consider hooking Claude up to a retrieval plugin for real sources. Also, specifying too many research steps might lead to verbose answers – balance thoroughness with brevity as required. Finally, remember that Claude will follow your lead; if you instruct it to assume something or you give it faulty context, it will build on that – ensure the initial info or approach is sound.

Snippet (User – research plan): *“Research this topic in a structured way. First, list what information we need and where to find it. Then systematically go through each point, and finally give a consolidated answer with references to the source of each point.”*

Snippet (User – encourage verification): *“Provide the answer, but verify each major fact against the data above. If you aren’t sure about something, state what would be needed to confirm it.”*

Snippet (User – multiple hypotheses): *“List two possible explanations for why the experiment failed, and what evidence in the report supports or contradicts each. Then state which explanation is better supported.”*

14. Handling Sensitive or Restricted Queries Safely

Value: Allows you to obtain useful answers on borderline sensitive topics by **framing the prompt to comply with safety guidelines**, rather than triggering a refusal. This technique is about being transparent and responsible with the AI so it doesn’t misunderstand your intent. Claude 4.5 is more tightly filtered (ASL-3) ³³, so a well-crafted prompt can mean the difference between a helpful answer and a generic refusal.

Mechanism: Acknowledge and clarify context. If your query involves potentially sensitive content (violence, medical, legal, etc.), preface it with a sentence that **sets boundaries and intent**. For example: *“This is for a fiction story that includes a crime scene; please handle it respectfully and avoid gore.”* By doing so, you show the AI you have a legitimate, safe purpose. Also, use neutral or clinical language for sensitive stuff – e.g. say “harm” instead of graphic descriptors, if detail isn’t needed. If Claude still refuses something that should be allowed, you can try the two-step approach from the creative writing example: ask it to outline or discuss abstractly first, then do the concrete task. E.g., *“Describe the approach to writing a scene about a war battle (no graphic detail, focus on strategy and emotion).”* Once it’s comfortable with that, you can proceed to writing the scene with those guidelines in mind. Always emphasize an educational, artistic, or helpful motive if applicable – Claude is more willing to engage if it’s clearly for a constructive purpose.

When to Use: When dealing with any content that skirts the edges of policy: violence, self-harm discussions, medical advice (where you must be careful to include “I am not a doctor, but...” type of context), legal scenarios, etc. Also if you find Claude responding with “I’m sorry, I can’t help with that,” but you believe the request is actually within allowed use, consider how to rephrase using this technique.

Pitfalls: Do not attempt to trick Claude into violating rules. That’s not what this is for – and it won’t work given the alignment improvements. The aim is to get valid responses on sensitive but legitimate queries. Always keep the AI’s limitations in mind: it’s not a lawyer or doctor, so even if it answers, use caution. Another pitfall is being too vague in reassurance – simply saying “don’t worry, it’s fine” is not effective. Instead, be specific about how you want the content handled. Lastly, if Claude does give a mild refusal or safe completion, you can adjust and try again, but repeated pushback might indicate the topic truly crosses a line (in which case, drop it).

Snippet (User – sensitive framing): *“I’m writing a historical fiction novel that involves a battle scene. I want to handle it tastefully and focus on the characters’ emotional experience rather than graphic details. Please help outline the scene. (No excessive gore, just the tension and aftermath.)”*

Snippet (User – medical context): “I understand you’re not a doctor, but I’d like some information about diabetes management for a school report. Please keep advice general and factual. For personalized guidance I’d consult a physician; here I just need an explanation of how insulin works.”

Snippet (User – legal context): “This is a hypothetical scenario for a study project, not real legal advice. Given that context: how would contract law handle a breach in a software agreement? What clauses typically cover this?”

Source Map

Source Title	URL	Author/ Organization	Date	Type	Techniques Supported (Keywords)
Anthropic Docs – Claude 4 Best Practices	https://docs.claude.com/.../claude-4-best-practices	Anthropic	2025-09-29	Official	Explicit instructions, verbosity balance, tool use patterns, CoT, context awareness, etc. ³ ⁷⁴
Anthropic Docs – Role Prompting	https://docs.claude.com/.../system-prompts	Anthropic	2025-09-30	Official	Role prompting, system vs user usage, examples of improved output ¹ ²
Anthropic Announcement – Introducing Claude 4.5	https://www.anthropic.com/news/claude-sonnet-4-5	Anthropic (News post)	2025-09-29	Official	New model capabilities (long tasks, coding, alignment) ³⁵ ³⁶ ; context window info ⁷⁵ ; safety (ASL-3) ⁷⁶
Reuters – Anthropic launches Claude 4.5	https://www.reuters.com/.../anthropic-launches-claude-45	J. Dastin & D. Seetharaman	2025-09-29	Media	Endurance (30h autonomy) ³⁷ ; agent focus; enterprise angle (Microsoft 365 integration) ⁵³
Dev.to – Claude 4.5 “What’s New & Limits”	https://dev.to/.../claude-sonnet-45-whats-new...	A0mine (Dev.to community)	2025-09-30	Community	Summary of new features & gotchas: agentic coding, context, pricing ⁴⁰ ⁴³ ; safety filter note ³³
Anthropic Engineering – Claude Code Best Practices	https://www.anthropic.com/engineering/claude-code-best...	Anthropic (Engineering)	2025-04-18	Official	(Pre-4.5 but relevant) Agentic coding tips: planning first ⁷ , using CLAUDE.md, correcting course ⁷⁷ , using /clear, etc.

Source Title	URL	Author/ Organization	Date	Type	Techniques Supported (Keywords)
Skywork.ai – Claude 4.5 for Creative Writing	https://skywork.ai/blog/claude-4-5-best-practices...	Andy Wang (Skywork AI)	2025-09-30	Community	Multi-pass workflow (outline→draft→revise) ¹⁰ ¹¹ ; context editing for creatives ⁷⁸ ; examples of style refinement ⁷⁹ ; pitfalls & fixes (verbosity, drift, etc.) ²¹ ⁹
AI Engineering Report – Claude Code 2.0 Review	https://aiengineering.report/p/claude-code-20-great-model	Bill Prin (Substack)	2025-10-01	Community	“Think harder”/“ultrathink” details ⁶ ; tab-to-think feature; critique of new Claude Code features. Confirms extended thinking triggers and model improvements.
Medium – 10 Game-Changing CLAUDE.md Entries	https://medium.com/...coding-superpower-eddf63f5ddf6	Reza Rezvani	2025-10-08	Community	Use of CLAUDE.md for persistent context (project brain) and examples of entries (style guidelines, commands) – enhances consistency across sessions. (Member-only article, summarized in analysis)
LessWrong – Claude 4.5 System Card Analysis	https://www.lesswrong.com/.../claude-sonnet-4-5-system...	LessWrong community	2025-10-02	Community	Notes on alignment: model is evaluation-aware; mentions that safety classifiers might flag content, requiring careful prompt wording. (Used for understanding ASL-3 impact, not directly quoted above.)

Source Title	URL	Author/ Organization	Date	Type	Techniques Supported (Keywords)
Reddit r/ ClaudeAI – User Discussions	https:// www.reddit.com/r/ ClaudeAI/...	Reddit users	Sep/Oct 2025	Community	Various practitioner tips: e.g. using “ultrathink” to force deeper reasoning, observations on Claude 4.5 failures and successes (provided anecdotal confirmation of official tips). (No single post fully cited, aggregated insights)
Anthropic Help Center – Extended Thinking	https:// support.claude.com/.../ using-extended- thinking	Anthropic Help	2025-09-29	Official	Documentation on extended thinking mode: how Claude can show a summarized chain-of-thought. Reinforces the “think” prompt usage.

(Sources above are referenced by their citation numbers throughout the report. Official sources are from Anthropic or partners, while community sources include individual experiments, blogs, and discussions that provide additional practical insights.)

Gaps & Open Questions

Despite the above best practices, **some open questions remain** in prompt engineering for Claude 4.5:

- **Optimal Use of 1M Token Context:** Claude 4.5 preview supports an unprecedented 1M-token context ⁸⁰. It’s unclear how performance scales with such huge inputs. *Open question: How to structure prompts when using 500K+ tokens of context?* (Does Claude maintain focus or does quality degrade? Are summaries or chunking still needed at that scale?) Real-world testing with near-limit contexts is needed to devise best practices for very large documents.
- **Extended Reasoning Limits:** We know keywords like “think harder” trigger deeper reasoning ⁶, but it’s not fully documented *how many levels* of “harder” there are (the system card hints at multiple levels). Also, **when might extended thinking be counterproductive?** If a task is simple, using ultrathink could waste time or even introduce confusion. Finding the boundary where extended thinking yields gains vs. when it’s overkill is an open area for experimentation.
- **Safety vs. Helpfulness Balance:** With stricter safety filters (ASL-3), Claude sometimes errs on the side of refusal ³³. We gave tips to reframe queries, but it’s not always predictable which borderline cases will trigger a refusal. For instance, some users report frustration with getting Claude to discuss benign but sensitive topics (like mental health scenarios). *Open question: Can we get more transparency or control over the safety tuning via prompts?* Currently, we rely on wording; future versions or settings might allow a more direct dialog about safety (e.g., “this

request is intended to be safe because...”). Until then, trial-and-error in phrasing is the tool, and it’s imperfect.

- **Subagent Orchestration Behaviors:** Claude 4.5 can spontaneously create sub-agents (tools) to delegate tasks ⁸¹, which is exciting but not fully predictable to users. We advise providing tools and letting it handle it, but *how do we debug when Claude’s agent-of-agent does something unexpected?* The internal decision to spawn a subagent isn’t exposed. It’s an open question how prompt engineers can **influence or inspect sub-agent reasoning**. Perhaps future updates or system cards will clarify this.
- **Consistency Over Very Long Sessions:** Claude is touted to maintain state over extremely long tasks, but practical limits exist (the 5-hour session reset, and weekly token caps for some users ⁷⁰). If you need a persistent conversational agent running for days, you’ll have to manually feed it its memory after resets. An open area is building a robust memory pipeline (storing state externally and re-injecting) – essentially, how to best do “conversation continuation” beyond the built-in window. There isn’t a one-click solution yet; it remains a custom engineering challenge.
- **Comparative Prompt Performance:** Some community prompts were developed for GPT-4 or other models. It’s not fully known if *all* those patterns work equally on Claude 4.5. For example, the classic “Let’s think step by step” works, but Claude might respond even better to slightly different wording given its training. There’s an opportunity for systematic testing: e.g., take a set of known effective GPT prompts (for reasoning, for creativity) and see if Claude 4.5 interprets or benefits from them differently. This could uncover Claude-specific optimizations (maybe it likes “Consider the following...” more than “Let’s think step by step,” hypothetically).
- **Tool Integration Nuances:** As new tools and plugins emerge (e.g. web browsing, database querying), how best to prompt Claude to use them effectively will be an ongoing learning process. Some patterns from Claude Code (like tagging certain text to indicate which tool to use) might generalize, but it’s an open question how well Claude 4.5 adapts to tools it wasn’t specifically trained on. Prompt engineers should be ready to iterate on “tool invocation prompts” to find what phrasing gets Claude to recognize when to use which tool.

Addressing these gaps will likely involve a combination of **community experimentation and future updates from Anthropic**. Prompt engineers should stay agile: Claude 4.5 is state-of-the-art now, but it’s evolving. Continuously testing assumptions, sharing findings, and updating best practices will be key to keeping the “Prompt Architect” agent (and this manual) effective over time.

Appendix: Notable Quotes from Sources

“Claude Sonnet 4.5 has a more concise and natural communication style... *Less verbose: May skip detailed summaries for efficiency unless prompted otherwise.*” ⁸²

“Using the system parameter to set Claude’s role... *the most powerful way to use system prompts with Claude*. The right role can turn Claude from a general assistant into your virtual domain expert!” ⁵⁷

“The straightforward way to get it to think harder is to simply use the words ‘**think harder**’ or ‘**ultrathink**’ in your prompt.” ⁶⁰

"Claude Sonnet 4.5 tends toward efficiency and *may skip verbal summaries after tool calls*, jumping directly to the next action." ²⁰

"In practice, Claude 4.5 performs best when you structure the collaboration into deliberate passes. *Below is a battle-tested sequence* that balances quality with speed..." ¹⁰

"Gotchas: *safety gating under ASL-3 may sometimes block benign content*" ³³ (Anthropic's stricter safety filters can flag normal requests; careful prompt wording can mitigate this).

"To make Claude more proactive about taking action by default... add this to your system prompt: `<default_to_action> By default, implement changes rather than only suggesting them... </default_to_action>`" ²⁴

"Use your thinking to plan and iterate based on this new information, and then take the best next action." ⁸³ (Guidance to Claude for interleaving reasoning with action, enabling complex problem solving).

¹ ² ⁵⁷ ⁵⁸ Giving Claude a role with a system prompt - Claude Docs

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/system-prompts>

³ ⁴ ⁵ ⁸ ¹³ ¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²² ²³ ²⁴ ²⁵ ²⁶ ²⁷ ²⁸ ²⁹ ³⁰ ³¹ ³² ⁴⁷ ⁴⁸ ⁵⁹ ⁶³ ⁶⁴ ⁶⁵ ⁶⁷

⁷² ⁷³ ⁷⁴ ⁸¹ ⁸² ⁸³ Claude 4 prompt engineering best practices - Claude Docs

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/claude-4-best-practices>

⁶ ⁶⁰ Claude Code 2.0 Is Promising But Flawed - by Bill Prin

<https://www.aiengineering.report/p/claude-code-20-great-model-but-flawed>

⁷ ⁶⁸ ⁶⁹ ⁷¹ ⁷⁷ Claude Code Best Practices \ Anthropic

<https://www.anthropic.com/engineering/claude-code-best-practices>

⁹ ¹⁰ ¹¹ ¹² ¹⁴ ²¹ ³⁴ ⁴⁴ ⁶¹ ⁶² ⁶⁶ ⁷⁸ ⁷⁹ ⁸⁰ Claude 4.5 Best Practices for Creative Writing, Brainstorming & Ideation (2025)

<https://skywork.ai/blog/claude-4-5-best-practices-creative-writing-brainstorming-ideation-2025/>

³³ ⁴⁰ ⁴¹ ⁴³ ⁴⁵ ⁵⁵ ⁷⁰ Claude Sonnet 4.5 — What's New and What Still Limits It (2025) - DEV Community

<https://dev.to/blamsa0mine/claude-sonnet-45-whats-new-and-what-still-limits-it-2025-42nj>

³⁵ ³⁷ ³⁸ ³⁹ ⁵³ ⁵⁴ Anthropic launches Claude 4.5, touts better abilities, targets business customers | Reuters

<https://www.reuters.com/business/retail-consumer/anthropic-launches-claude-45-touts-better-abilities-targets-business-customers-2025-09-29/>

³⁶ ⁴² ⁴⁶ ⁴⁹ ⁵⁰ ⁵¹ ⁵² ⁵⁶ ⁷⁵ ⁷⁶ Introducing Claude Sonnet 4.5 \ Anthropic

<https://www.anthropic.com/news/claude-sonnet-4-5>